

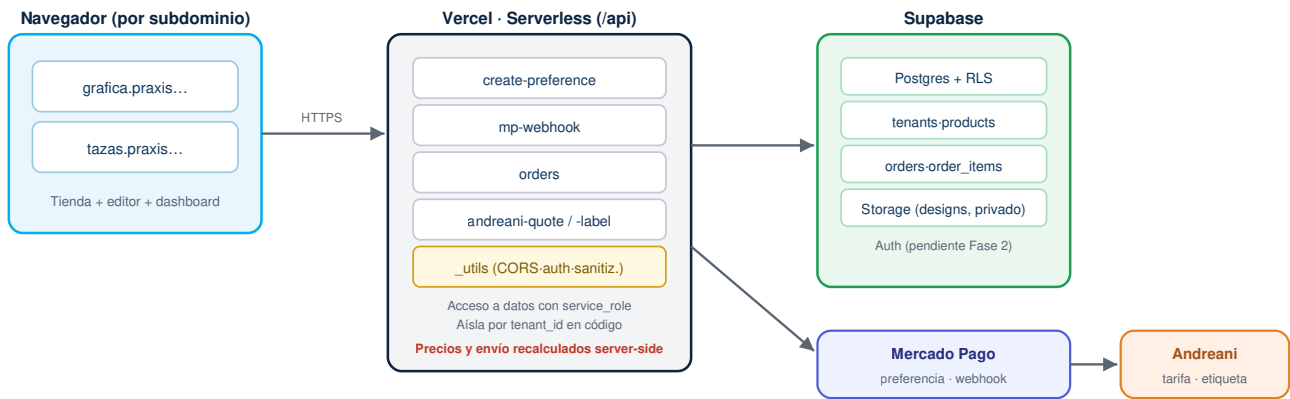
# Praxis Personaliza

Arquitectura del sistema · Fortalezas y debilidades · Estado del desarrollo

SaaS de ecommerce para productos personalizables · Fase 1 (auditoría y endurecimiento) · Para: Isaac (co-desarrollador)

Praxis Personaliza es una plataforma **multitenant**: una sola base de código y una sola base de datos sirven a varios comercios. “A Tu Manera Gráfica” es solo un tenant. Este documento refleja el estado luego de la Fase 1, donde se auditó el sistema y se corrigió lo corregible sin reescribir la autenticación.

## 1. Arquitectura general



**Stack:** frontend estático (HTML/JS) servido por Vercel · funciones serverless en /api (Node) · Supabase (Postgres + Storage) · Mercado Pago (pagos) · Andreani (envíos, Argentina). El tenant se resolverá por subdominio (decisión de arquitectura de esta fase).

## 2. Modelo de datos

| Tabla       | Rol                                                             | Aislamiento por tenant                             |
|-------------|-----------------------------------------------------------------|----------------------------------------------------|
| tenants     | Un comercio: marca, colores, credenciales, config de envío.     | Raíz de todo.                                      |
| products    | Catálogo por comercio. Precio en centavos.                      | tenant_id NOT NULL · slug único <i>por</i> tenant. |
| orders      | Pedidos: comprador, envío, pago, estado.                        | tenant_id NOT NULL.                                |
| order_items | Ítems + diseño personalizado (texto, fuente, color, miniatura). | Vía order_id → orders.tenant_id.                   |

### 3. Fortalezas y debilidades

#### Fortalezas

- **Precios y envío se recalculan en el servidor:** no se confía en el carrito del navegador (anti-manipulación).
- El **webhook de pago verifica** que el monto pagado coincida con el total del pedido.
- Validación y sanitización **centralizada** en `_utils` (email, teléfono, montos, UUID).
- CORS por origen permitido y auth admin con comparación **tiempo-constante**.
- Serverless: **escala horizontal** automática; base gestionada.
- Esquema **idempotente y versionado**; advisors de seguridad en `0`.

#### Debilidades / riesgos

- **Sin autenticación de usuarios real:** hoy un único token global de admin.
- **Pagos y envíos con una sola cuenta** para todos los tenants (ver F1/F2).
- Acceso a datos 100% con `service_role`: la **RLS no protege**; el aislamiento vive en el código.
- Frontend monolítico (`index.html`, ~2.100 líneas) con base64 embebido: difícil de mantener.
- Miniaturas de diseño **en la base** (base64), no en storage.
- **Sin tests automatizados.**

### 4. Hallazgos de la auditoría

| ID | Severidad | Hallazgo                                                                                                                                                                       | Estado                                                                                             |
|----|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| F1 | CRÍTICO   | <b>Un solo Mercado Pago para todos los tenants.</b><br>MP_ACCESS_TOKEN global +<br>MP_PUBLIC_KEY hardcodeada en el front → todo comercio cobraría en la cuenta de A Tu Manera. | <b>Fase 2</b> credenciales MP por tenant                                                           |
| F2 | CRÍTICO   | <b>Un solo Andreani para todos.</b><br>ANDREANI_USER/<br>PASS globales → etiquetas con la cuenta de A Tu Manera.                                                               | <b>Fase 2</b> credenciales por tenant                                                              |
| F3 | CRÍTICO   | <b>Aislamiento apoyado en un header falsificable</b> (X-Tenant) + token admin único; el código caía a “todos los tenants” si el tenant no resolvía.                            | <b>Mitigado F1</b> se exige tenant válido y se filtra siempre; falta derivar tenant del login/host |

|     |       |                                                                                                            |                                                                     |
|-----|-------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| F4  | ALTO  | Acceso a DB solo con <code>service_role</code> (bypass total de RLS).                                      | Fase 2 Auth + RLS por tenant                                        |
| F5  | ALTO  | <code>products.slug</code> era único <i>global</i> → dos tenants no podían compartir slug.                 | Corregido único por (tenant_id, slug)                               |
| F6  | ALTO  | <code>tenant_id</code> era nullable → pedidos/ productos “huérfanos”.                                      | Corregido NOT NULL                                                  |
| F7  | MEDIO | Miniaturas base64 (hasta ~250 KB/ítem) en la columna → infla <code>order_items</code> y el dashboard.      | Bucket privado · Fase 2 reconectar flujo con URL firmada            |
| F8  | MEDIO | Pesos/volumenes de envío hardcoded por nombre de producto → no escala a productos/tenants nuevos.          | Fase 2 columnas <code>weight_grams</code> / <code>volume_cm3</code> |
| F9  | MEDIO | <code>FALLBACK_PRODUCTS</code> hardcodeaba productos de A Tu Manera → fuga entre tenants y precios viejos. | Eliminado                                                           |
| F10 | BAJO  | <code>order_number</code> es serial global (no por tenant): numeración no contigua por comercio.           | Opcional                                                            |
| F11 | BAJO  | XSS en el dashboard: los datos del comprador se renderizan con <code>escapeHTML()</code> .                 | OK · revisar 1–2 spots del editor                                   |

**Acciones ya ejecutadas en base (Security First):** se removieron políticas RLS de INSERT público irrestricto (`orders`, `order_items`); se cerró el bucket `designs` (sin upload/listado público, ahora privado); se fijó `search_path` del trigger; se forzó `tenant_id NOT NULL`; se reemplazó el unique global de slug por unique por-tenant. Advisors de seguridad de Supabase: **0 avisos**.

## 5. Aislamiento de tenants — estado

| Dimensión                 | Hoy                                                                          | Objetivo (Fase 2)                                                       |
|---------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Datos (catálogo, pedidos) | Acotado por tenant_id en <i>todos</i> los endpoints; se exige tenant válido. | + RLS efectiva por tenant como segunda barrera.                         |
| Identidad del tenant      | Header X-Tenant (se endureció, pero es cliente).                             | Derivada del <b>subdominio</b> y del <b>login</b> , no del cliente.     |
| Dinero (pagos)            | Compartido — riesgo                                                          | Cuenta Mercado Pago <b>por tenant</b> .                                 |
| Envíos                    | Compartido — riesgo                                                          | Cuenta Andreani <b>por tenant</b> .                                     |
| Administración            | Un token global de admin.                                                    | Superuser + admins por tenant (Supabase Auth + rol + tenant en el JWT). |

## 6. Estado del desarrollo y recomendación

**Dónde estamos:** un MVP funcional que recién pasó de mono-tenant a base multitenant. Para el tenant A Tu Manera Gráfica funciona el circuito completo: tienda, editor de diseño, checkout con Mercado Pago, dashboard de pedidos y generación de etiqueta Andreani. En Fase 1 se puso y endureció la infraestructura multitenant (tabla tenants, tenant\_id, aislamiento en código, hardening de base).

**Qué se puede hacer hoy:** operar A Tu Manera de punta a punta — vender, cobrar y despachar.

**Qué todavía NO:** dar de alta un segundo comercio que cobre en *su* cuenta; que un vendedor administre su catálogo con *su* login; publicar una landing por tenant. Todo eso depende de la autenticación real y las credenciales por tenant.

**Recomendación — seguir por fases:**

| Fase | Foco                                                                                                                                                                                                                                                        |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2    | Supabase Auth (superuser + admins por tenant, tenant y rol en el JWT) · credenciales MP y Andreani <b>por tenant</b> · RLS efectiva · panel superuser (alta/config de tenants y usuarios) · segundo tenant de prueba (tazas) · migrar miniaturas a storage. |
| 3    | Landing pages por tenant (template configurable: hero, productos, secciones) · branding por tenant desde tenants · manual del vendedor (PDF) documentando el dashboard ya construido.                                                                       |

Documento generado en Fase 1. La numeración de hallazgos (F1–F11) se mantiene estable para referencia entre documentos y commits.